

6. Comunicação Baseada em Chamadas Remotas: RPC e gRPC

Objetivos de aprendizagem

- Explicar abstração RPC, stubs, IDL, serialização e marshalling.
- Definir serviços com Protocol Buffers.
- Gerar e executar cliente e servidor gRPC em Python.
- Usar deadlines/timeouts e interpretar status de erro.
- Comparar acoplamento de contrato de gRPC com REST e MQTT.

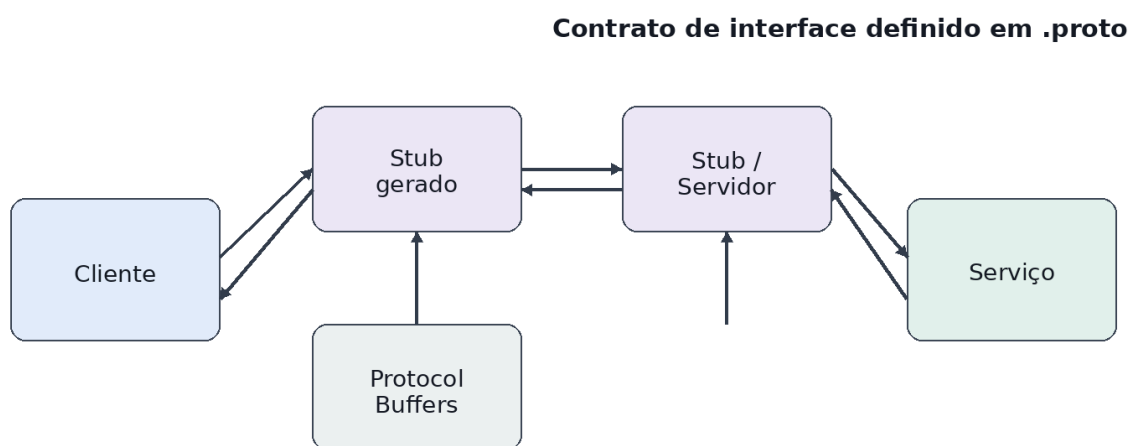


Figura 5 - Fluxo conceitual de uma chamada gRPC com stubs e Protocol Buffers

6.1 A ilusão de chamada local

RPC permite que um programa invoque uma operação remota por uma interface semelhante a uma chamada de procedimento. Essa abstração é produtiva, mas incompleta: a chamada atravessa a rede, portanto possui latência, serialização e falhas que não existem em uma função local. O programador deve tratar deadlines, indisponibilidade e semântica de retry.

6.2 IDL e geração de stubs

Em gRPC, o contrato costuma ser definido em um arquivo .proto. A IDL descreve mensagens e métodos; ferramentas geram código cliente e servidor. Isso produz forte contrato de tipos e boa interoperabilidade entre linguagens, ao custo de maior acoplamento ao esquema e necessidade de gestão de evolução.

6.3 Tipos de RPC em gRPC

- Unary: uma requisição e uma resposta.
- Server streaming: uma requisição produz fluxo de respostas.
- Client streaming: fluxo de requisições produz uma resposta.
- Bidirectional streaming: ambos enviam fluxos independentemente.

6.4 Laboratório

Terminal

```
pip install grpcio grpcio-tools
```

processador.proto

```
syntax = "proto3";

package laboratorio;

service Processador {
  rpc ConverterTemperatura (TemperaturaRequest) returns (TemperaturaResponse);
  rpc Media (MediaRequest) returns (MediaResponse);
}

message TemperaturaRequest {
  double valor = 1;
  string origem = 2;
  string destino = 3;
}

message TemperaturaResponse {
  double valor = 1;
  string unidade = 2;
}

message MediaRequest {
  repeated double valores = 1;
}

message MediaResponse {
  double media = 1;
  int32 quantidade = 2;
}
```

Terminal

```
python -m grpc_tools.protoc -I. --python_out=. --grpc_python_out=. processador.proto
```

servidor.py

```
from concurrent import futures
import time
import grpc
import processador_pb2 as pb2
import processador_pb2_grpc as pb2_grpc

class Processador(pb2_grpc.ProcessadorServicer):
    def ConverterTemperatura(self, request, context):
        origem = request.origem.upper()
        destino = request.destino.upper()
        if origem == destino:
            valor = request.valor
        elif origem == "C" and destino == "F":
            valor = request.valor * 9 / 5 + 32
        elif origem == "F" and destino == "C":
            valor = (request.valor - 32) * 5 / 9
        else:
            context.abort(grpc.StatusCode.INVALID_ARGUMENT, "conversão não suportada")
        return pb2.TemperaturaResponse(valor=valor, unidade=destino)

    def Media(self, request, context):
        if not request.valores:
            context.abort(grpc.StatusCode.INVALID_ARGUMENT, "lista vazia")
        # atraso opcional para experimentar deadline
        if len(request.valores) > 1000:
            time.sleep(2)
        return pb2.MediaResponse(
            media=sum(request.valores) / len(request.valores),
```

```

        quantidade=len(request.valores),
    )

def serve():
    server = grpc.server(futures.ThreadPoolExecutor(max_workers=10))
    pb2_grpc.add_ProcessadorServicer_to_server(Processador(), server)
    server.add_insecure_port("[::]:50051")
    server.start()
    print("Servidor gRPC em :50051")
    server.wait_for_termination()

if __name__ == "__main__":
    serve()

```

cliente.py

```

import grpc
import processador_pb2 as pb2
import processador_pb2_grpc as pb2_grpc

with grpc.insecure_channel("localhost:50051") as channel:
    stub = pb2_grpc.ProcessadorStub(channel)

    try:
        r = stub.ConverterTemperatura(
            pb2.TemperaturaRequest(valor=25, origem="C", destino="F"),
            timeout=1.0,
        )
        print(r.valor, r.unidade)
    except grpc.RpcError as exc:
        print("RPC falhou:", exc.code(), exc.details())

    try:
        valores = list(range(2000))
        r = stub.Media(pb2.MediaRequest(valores=valores), timeout=0.5)
        print(r.media)
    except grpc.RpcError as exc:
        print("deadline/erro:", exc.code(), exc.details())

```

6.5 Deadlines, timeouts e cancelamento

Deadline define quanto tempo o cliente aceita esperar. Sem limite, uma chamada pode bloquear recursos por tempo indeterminado. O servidor também pode observar cancelamento e interromper trabalho que já não interessa ao cliente. Em sistemas compostos por múltiplos serviços, o orçamento de tempo deve ser propagado cuidadosamente para evitar que cada hop use um timeout completo e ultrapasse o SLA total.

6.6 Evolução de contrato

Protocol Buffers usa números de campo para compatibilidade. Campos removidos não devem ter seus números reutilizados. Novos campos opcionais podem ser adicionados preservando clientes antigos. Mudanças incompatíveis exigem versionamento ou migração coordenada. O contrato forte reduz ambiguidades, mas torna a disciplina de evolução essencial.

6.7 Segurança

O exemplo usa canal inseguro para laboratório local. Em ambientes reais, gRPC pode usar TLS e autenticação. Também é necessário controlar autorização por método, tamanho de mensagens, limites de concorrência e exposição de metadados.

6.8 Experimentos

- Execute o cliente com o servidor desligado e registre o status gRPC.
- Adicione sleep em um método e compare deadlines de 100 ms, 1 s e 3 s.

- Envie argumento inválido e diferencie INVALID_ARGUMENT de UNAVAILABLE.
- Aumente max_workers e execute múltiplos clientes concorrentes.
- Adicione um novo campo ao .proto e verifique compatibilidade entre versões geradas.

AP4 - Serviço remoto com gRPC

Atividade prática

Implemente um serviço gRPC com múltiplas operações e realize experimentos de erro, deadline, concorrência e evolução de contrato.

Objetivos da atividade

- Modelar contrato com .proto.
- Gerar stubs e implementar servidor e cliente.
- Tratar erros de domínio com status gRPC.
- Demonstrar deadlines e indisponibilidade.

Requisitos obrigatórios

- Pelo menos três métodos RPC.
- No mínimo três tipos de mensagem próprios.
- Validação explícita e ao menos dois status de erro.
- Cliente com deadline configurado.
- Experimento concorrente com múltiplas chamadas.

Roteiro mínimo

1. Defina o contrato antes da implementação.
2. Gere stubs de forma reproduzível.
3. Implemente métodos e testes.
4. Provoque indisponibilidade e deadline.
5. Altere o contrato de forma compatível e documente o resultado.

Entregáveis

- Arquivo .proto e código gerado/gerável.
- Servidor e cliente.
- Script de teste ou roteiro reproduzível.
- Tabela de latência/status por experimento.
- Análise comparando gRPC com a AP2 e AP3.

Questões para análise

1. Por que uma chamada remota não deve ser tratada como função local?
2. Qual diferença entre erro de aplicação e indisponibilidade do serviço?
3. O contrato forte aumenta qual tipo de acoplamento?
4. Como você faria retry sem duplicar efeitos perigosos?
5. Que mudança no .proto seria incompatível?

Critérios de avaliação

Tabela 11 - Critérios de avaliação da AP4

Critério	Evidência esperada
Contrato	IDL clara, tipos adequados e evolução consciente.
Implementação	Servidor/cliente corretos e tratam erros.
Resiliência	Deadlines e falhas observados.
Análise comparativa	Discussão relaciona gRPC, REST e MQTT.
Demonstração	Execução reproduzível.

Checklist antes da apresentação

- ☐ O .proto pode ser regenerado do zero.
- ☐ O cliente define deadline.
- ☐ Existem testes de erro e servidor indisponível.
- ☐ Há experimento concorrente.
- ☐ Foi realizada mudança compatível no contrato.

Desafios opcionais

- Implemente streaming.
- Adicione TLS local.
- Implemente interceptor para correlação e métricas.

6.9 Streaming e fluxo contínuo

Streaming em gRPC evita abrir uma nova RPC para cada item e permite fluxo de mensagens em uma conexão lógica. Server streaming é útil para resultados incrementais; client streaming para agregação; bidirecional para sessões interativas. O uso de streaming, contudo, mantém recursos ocupados por mais tempo e exige estratégias para desconexão, retomada e controle de fluxo.

6.10 Status e modelo de erro

gRPC possui códigos de status padronizados. INVALID_ARGUMENT indica erro independente do estado do sistema; NOT_FOUND indica entidade ausente; ALREADY_EXISTS e FAILED_PRECONDITION expressam conflitos; UNAVAILABLE tende a indicar falha transitória de infraestrutura; DEADLINE_EXCEEDED indica que o tempo foi esgotado. Escolher status coerentes permite que clientes decidam se devem corrigir entrada, desistir ou repetir.

Retry seletivo

UNAVAILABLE pode ser transitório, mas retry ainda depende da operação ser segura para repetição. INVALID_ARGUMENT não melhora com retry. DEADLINE_EXCEEDED é ambíguo quanto a efeitos já executados no servidor. A política deve conhecer semântica de negócio.

6.11 Interceptors e preocupações transversais

Interceptors podem adicionar logging, autenticação, métricas ou correlação sem duplicar código em cada método. Esse padrão é poderoso, mas deve evitar esconder regras de negócio. Observabilidade transversal é apropriada; uma decisão de domínio importante costuma pertencer ao serviço explícito.

6.12 Health checking e balanceamento

Uma plataforma precisa decidir para quais réplicas enviar chamadas. Health checks indicam se a instância está apta a receber tráfego. Um processo vivo não é necessariamente saudável: pode estar sem acesso a dependência crítica ou com fila saturada. Diferencie liveness, readiness e indicadores de degradação. Em clientes gRPC, resolução de nomes e balanceamento podem ser integrados à infraestrutura.